A REPORT ON

Text Editor

Submitted in partial fulfilment of the requirement for the award of the degree of BACHELOR OF COMPUTER APPLICATION

Submitted by: Mayank Bisht
Sec. - A (G2)
BCA (III)
Roll No.: 68

Project Number : M11

Under the Guidance of Dr. Jitendra Chaudhary, Associate Professor, School of Computing

Graphic Era Hill University, Bhimtal, Uttarakhand March-2025

Graphic Era Hill University
BHIMTAL CAMPUS

CANDIDATE'S DECLARATION

I/We hereby certify that the work which is being presented in the Synopsis entitled **"Text Editor"** in partial fulfilment of the requirements for the award of the Degree of Bachelor Computer Application of the Graphic Era Hill University, Bhimtal shall be carried out by the undersigned under the supervision of **Dr. Jitendra Chaudhary, Associate Professor**, School of Computing, Graphic Era Hill University, Bhimtal.

Mayank Bisht	2471122	signature

The above-mentioned students shall be working under the supervision of the undersigned on the **"Text Editor"**.

Signature	Signature
Supervisor	**Head of the Department**

Table of Contents

Chapter No.	Description	Page No.
Chapter 1	• Introduction and Problem Statement	4
Chapter 2	• Background / Literature Survey	5
Chapter 3	• Objectives	6
Chapter 4	• Hardware and Software Requirements	7
Chapter 5	• Possible Approach / Algorithms	8-9
Chapter 6	• Data Structures Used	10
Chapter 7	• Challenges	11
	• Flow chart	12
	• Output	13-14
	• References & Conclusion	15
	• Acknowledgment	16

Chapter 1: Introduction and Problem Statement

The advancement of web technologies has made it possible to develop powerful applications directly within the browser. One such application is a **Text Editor Website**, which provides an efficient way to create, edit, and format text without depending on bulky desktop applications. This project focuses on building a simple yet practical text editor using React.js, ensuring accessibility, speed, and ease of use.

The **introduction** of the project lies in addressing the needs of modern users who require quick editing tools without installing heavy software. Tools like Notepad are too limited, while advanced platforms like Google Docs or MS Word are overloaded with features that are unnecessary for basic use. Users often face challenges when they just want an editor with essential formatting functions in a lightweight environment.

The **problem statement** therefore centers on the lack of an efficient, browser-based solution that combines simplicity with usability. While existing solutions either compromise on features or overload the user, this project seeks to provide a middle-ground alternative. By leveraging React's component-based architecture, the project ensures flexibility, modularity, and scalability, making it adaptable for future improvements.

Thus, the **purpose** of the project is to provide a user-friendly text editing platform that is accessible anytime, anywhere, and meets the basic requirements of students, professionals, and casual users alike. This sets the foundation for further exploration into web-based productivity tools.

Chapter 2: Background/ Literature Survey

Several **existing text editors** were studied to understand their design and features. Applications like **Notepad** and **WordPad** are widely used for basic editing, but they lack advanced formatting. On the other hand, platforms such as **Google Docs** and **Microsoft Word** provide robust features but require high system resources and can overwhelm users. The literature shows that there is a gap for an editor that is both lightweight and functional.

Research into **web technologies** revealed that React.js is one of the most suitable frameworks for this project. It provides reusability of components, efficient state management, and high performance, which are crucial for building real-time editing applications. Articles from MDN and W3Schools highlight React's virtual DOM and component-based architecture as key features for interactive web development.

From a user perspective, the trend has shifted toward **online tools** that do not require installations and can be accessed across devices. This is evident from the growing popularity of cloud-based applications. However, many of these require continuous internet access and sometimes user accounts, which limit accessibility.

The **literature survey** concludes that there is scope for creating an open, lightweight, and accessible text editor that can bridge the gap between simplicity and complexity. This motivates the development of a React-based editor that fulfills these needs while remaining scalable.

Chapter 3: Objectives

The **main objective** of the Text Editor Website project is to create a browser-based tool that is easy to use, accessible, and responsive. The project aims to deliver an editor that balances the simplicity of Notepad with the flexibility of advanced editors.

Some of the **specific objectives** include:

- Providing formatting options such as bold, italic, underline, font size, and text coloring.
- Implementing **dark and light themes** to enhance user experience.
- Ensuring cross-device compatibility, so that the editor works on desktops, laptops, and mobile browsers.
- Keeping the application **lightweight** by using React's component reusability and efficient state handling.

Another important objective is to design the system in a way that it remains **scalable**. This means new features such as exporting text as PDF/Word, collaborative editing, and AI-assisted suggestions can be added without rebuilding the entire application.

Overall, the project seeks to create an editor that is not only useful in its current state but also adaptable for future use cases, making it a versatile tool for a variety of users.

Chapter 4: Hardware and Software Requirements

Hardware Requirements:

- Minimum 4 GB RAM
- Dual-core processor (Intel/AMD/Apple M1 or higher)
- 500 MB free storage
- Internet connectivity for deployment and testing

Software Requirements:

- Operating System: Windows / macOS / Linux
- Node.js and npm installed
- React.js library and supporting packages
- Code Editor: VS Code (or equivalent)
- Web Browser: Google Chrome / Safari / Edge

Chapter 5: Possible Approach/ Algorithms

The project follows a **modular approach** where the text editor is built as a React application using reusable components. The editor uses **state management** in React to handle real-time text changes.

Algorithm (Simplified Steps):

1. Initialize a React project and set up editor components.
2. Capture user input using controlled text fields.
3. Apply formatting functions (bold, italic, underline, font size, color) using state and event handling.
4. Store user preferences (like theme selection) in local storage for persistence.
5. Render updated text dynamically on the screen.
6. (Optional) Implement undo/redo feature using a stack-based approach.

This approach ensures smooth interaction, minimal lag, and efficient use of browser resources.

Algorithm:

1. **Start / Initialize**

    - **Read saved state from localStorage (content, theme).**
    - **Initialize React state: content, selection, formatState, theme, undoStack, redoStack.**
    - **Render UI: Toolbar (Bold/Italic/Underline/Size/Color), Theme switch, Editor area, Counters, Export buttons.**

2. **User Input Loop**

    - **On every keystroke or paste, update content.**
    - **Push previous state to undoStack (debounced to avoid spam).**
    - **Clear redoStack.**

3. **Formatting**

    - **When user clicks a format button or presses a shortcut (e.g., Ctrl+B), read current selection.**
    - **Apply format to selected text (rich text via contentEditable + Range API, or plain text rules).**
    - **Update content and formatState, then record history.**

4. **Autosave & Persistence**

    - **Debounce autosave (e.g., 500 ms after last change) → store content + theme in localStorage.**

5. **Undo/Redo**

    - **Undo pops from undoStack → pushes current state onto redoStack → sets content.**
    - **Redo pops from redoStack → pushes current state onto undoStack → sets content.**

6. **Utilities**

    - **Theme toggle: flip theme, persist to localStorage, re-render.**
    - **Stats: compute word/char count on content change.**
    - **Export: create a Blob from content (TXT/HTML) and trigger download.**

7. **End**

    - **On unmount/close, ensure latest autosave flushed.**

Chapter 6: Data Structures Used

The app's data model is lightweight and functional: most runtime state is held in component-level React state (strings like text, mode, and small objects like the alert { msg, type } in App.js), with controlled form fields (email/message) using local state for input handling; text processing uses transient arrays (from String.prototype.split) and array methods (map, filter, sort, reduce) for transformations, a Set to deduplicate words, and regular expressions for pattern matching and replacements (removing punctuation, numbers, extra spaces, emojis, etc.).

Component communication relies on props and callbacks (for example, showAlert is passed from App to TextForm), navigation state is handled by react-router <Routes>/<Route>, and UI-only values (styles, button disabled flags) are derived from mode and computed inline style objects rather than a global store. All text operations are effectively immutable (they produce new strings/arrays), which simplifies reasoning and avoids shared mutable state; ephemeral operations like copying use browser APIs (clipboard) and no persistence or database is involved.

This design favors simplicity and browser-side privacy but can be extended later with a centralized store (Context or Redux) if cross-component state grows, or with memoization to optimize expensive transforms on large texts.

Challenges

1. The development build surfaced tooling and linting issues (ESLint warnings, incorrect JSX attributes, and broken imports) which interrupted iterative testing. Resolving these required combing through components to replace invalid attributes (for → htmlFor, class → className), removing or wiring unused handlers, and adjusting component APIs so props and hooks behaved predictably. That work improved code quality and reduced noisy compiler output, making regressions easier to spot.

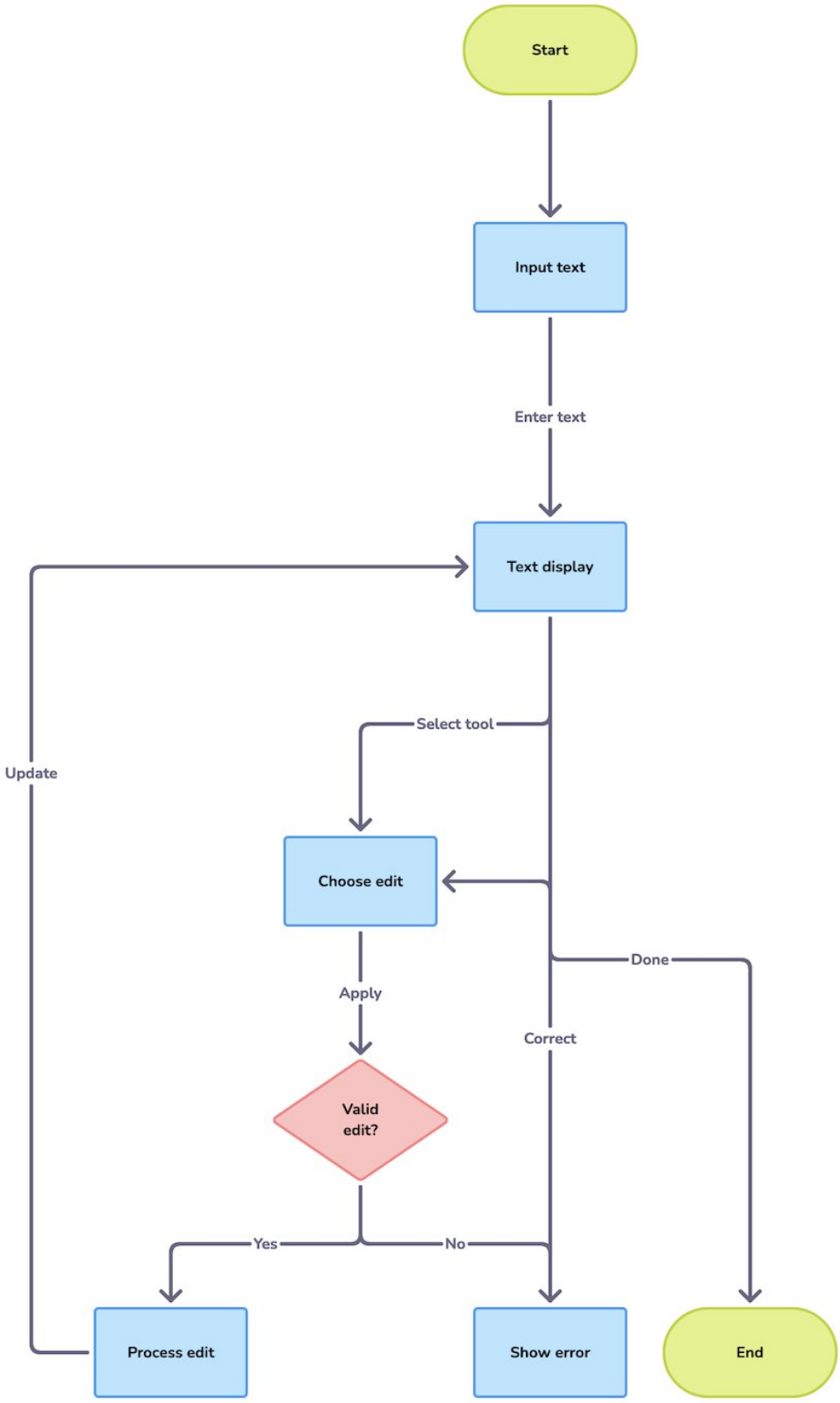
2. Implementing robust UI behavior revealed several edge cases in state and input handling. Users pasting only whitespace, unusual emoji/Unicode sequences, or large bodies of text caused inconsistent button states and fragile string operations. I hardened the code by normalizing input (trim checks), using defensive mapping for word transformations, and relying on RegExp patterns and Sets for reliable deduplication and filtering, which improved UX and reduced runtime errors.

3. Dark-mode theming and styling consistency were non-trivial to get right across components. Some styles were hard-coded (e.g., footer background) or used different mechanisms (inline style objects vs. Bootstrap classes), producing visual inconsistencies. I standardized theme-dependent styling (mode-driven class selection and helper functions) and made interactive controls (theme toggle) controlled components so the UI state always reflects the underlying mode, improving accessibility and visual coherence.

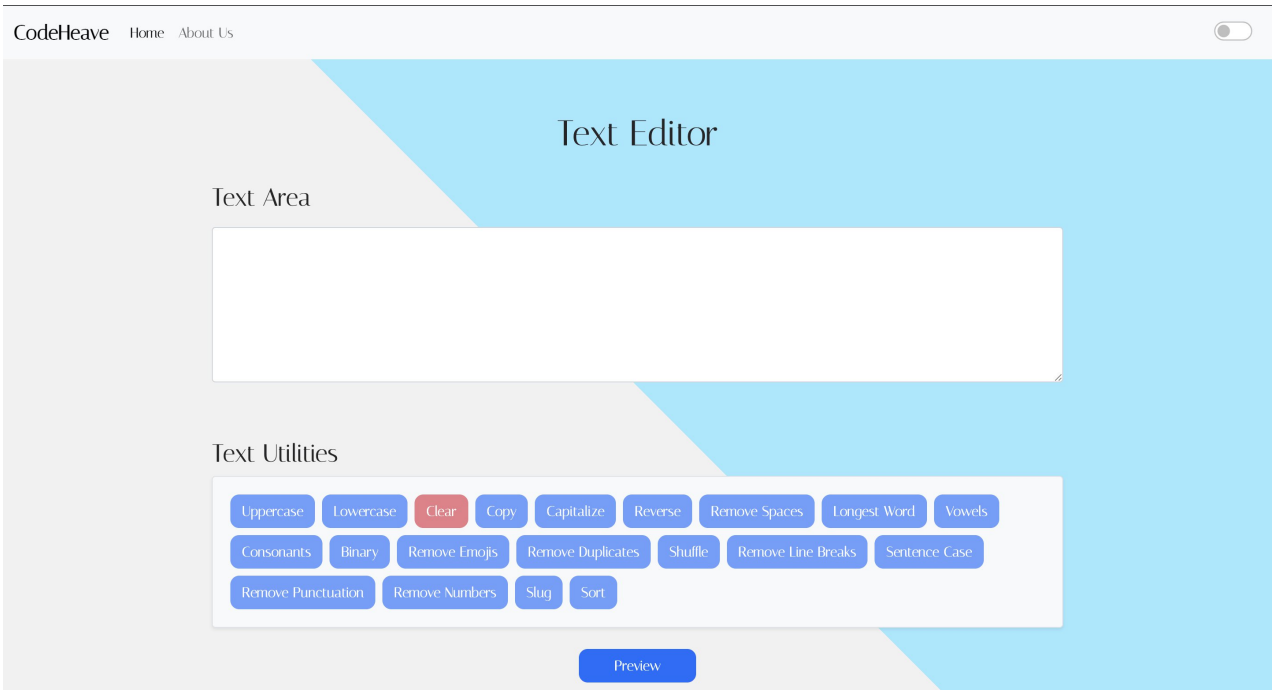
4. Security, compatibility and maintenance concerns surfaced while installing packages: audit warnings, deprecated transitive dependencies, and engine requirement mismatches. While most issues were non-blocking, they indicate technical debt and potential vulnerabilities. The long-term remedy is dependency upgrades, replacing deprecated packages, and adopting automated dependency monitoring and periodic audits.

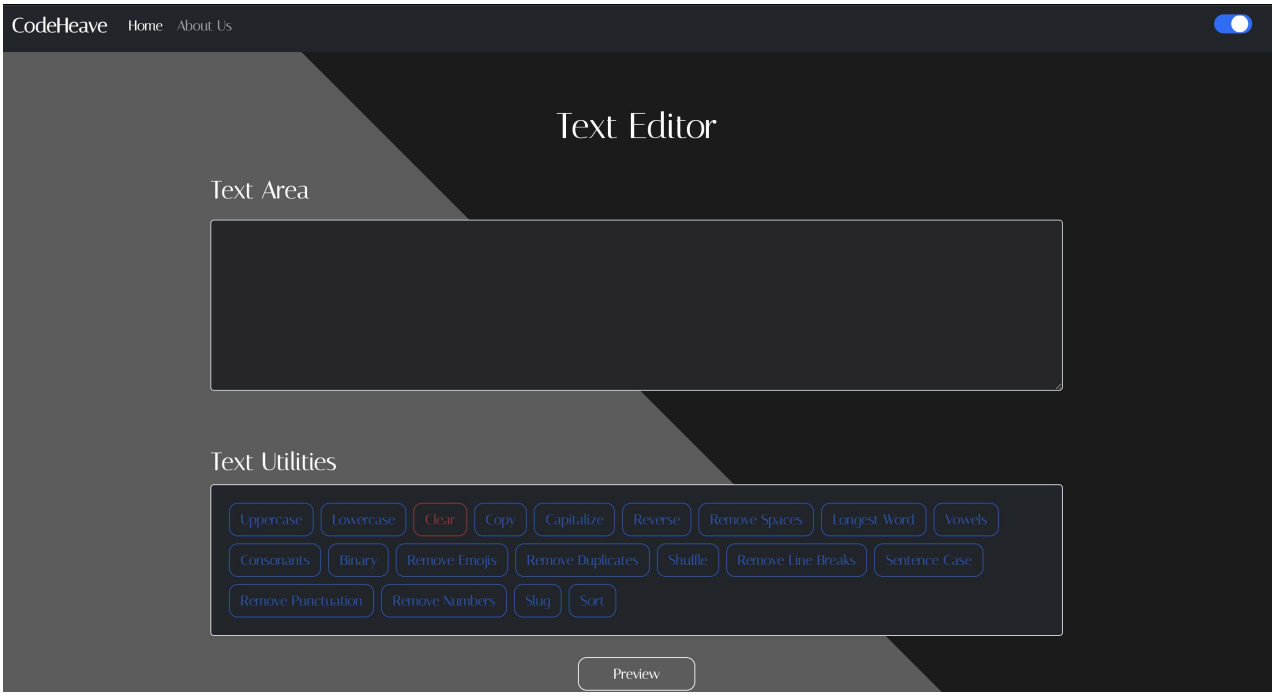
5. Finally, performance and testability considerations emerged when thinking about scaling the app. Several text transforms are O(n) and could be costly on very large inputs; moreover, the app lacks unit tests around key transformations. Mitigations include adding input-size guards, memoizing expensive computations, and introducing automated tests for handler logic to prevent regressions.

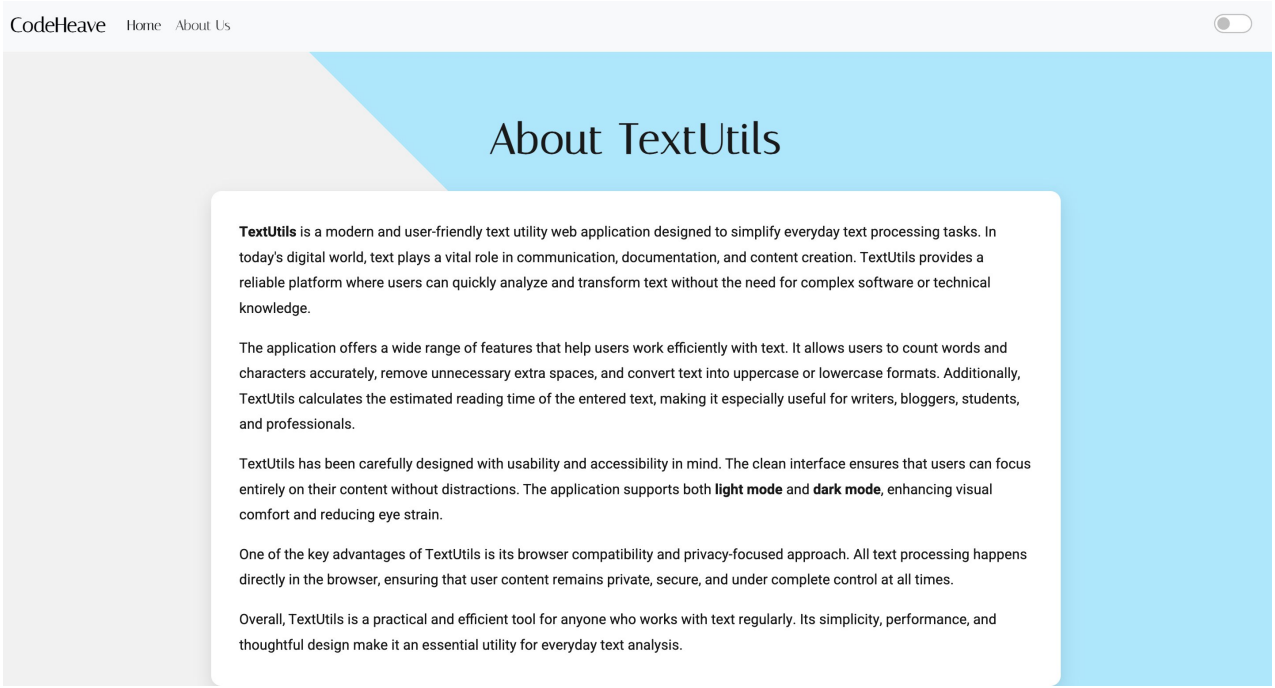
Flow Chart

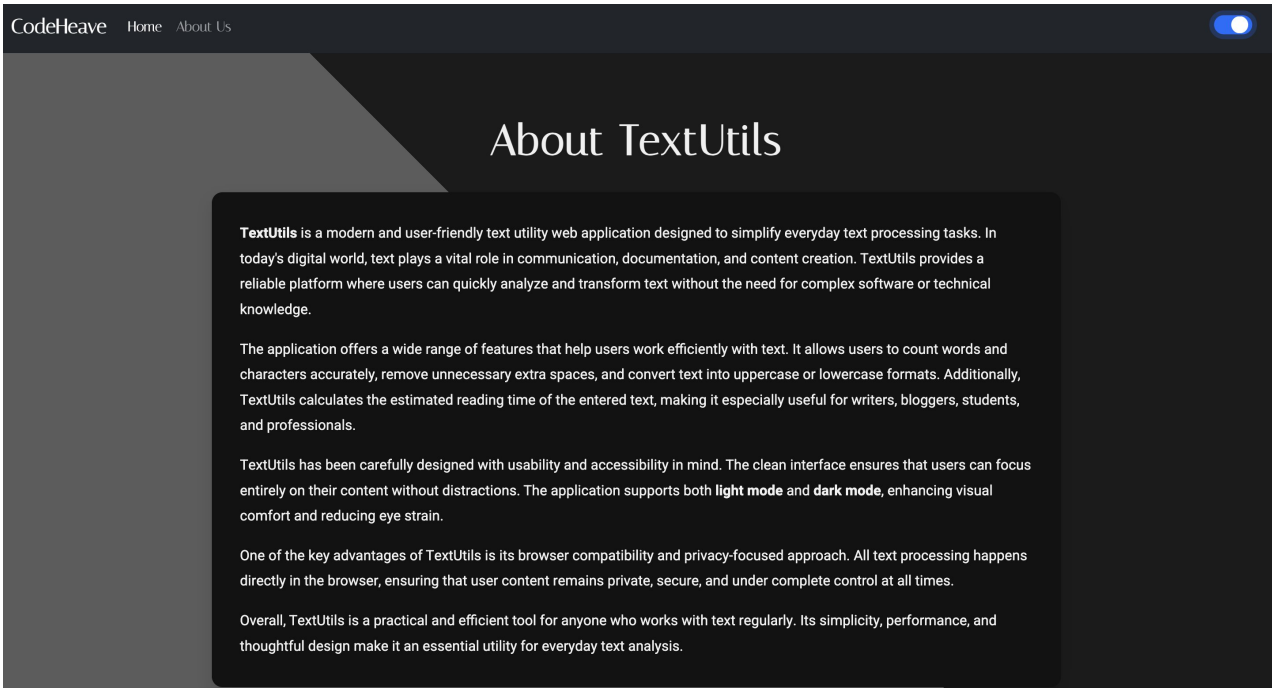

Start
Input text
Enter text
Text display
Select tool
Update
Choose edit
Done
Apply
Correct
Valid edit?
Yes
No
Process edit
Show error
End

Output


CodeHeave
Home
About Us
Text Editor
Text Area
Text Utilities
Uppercase
Lowercase
Clear
Copy
Capitalize
Reverse
Remove Spaces
Longest Word
Vowels
Consonants
Binary
Remove Emojis
Remove Duplicates
Shuffle
Remove Line Breaks
Sentence Case
Remove Punctuation
Remove Numbers
Slug
Sort
Preview


CodeHeave
Home
About Us
Text Editor
Text Area
Text Utilities
Uppercase
Lowercase
Clear
Copy
Capitalize
Reverse
Remove Spaces
Longest Word
Vowels
Consonants
Binary
Remove Emojis
Remove Duplicates
Shuffle
Remove Line Breaks
Sentence Case
Remove Punctuation
Remove Numbers
Slug
Sort
Preview


CodeHeave
Home
About Us
About TextUtils
TextUtils is a modern and user-friendly text utility web application designed to simplify everyday text processing tasks. In today's digital world, text plays a vital role in communication, documentation, and content creation. TextUtils provides a reliable platform where users can quickly analyze and transform text without the need for complex software or technical knowledge.
The application offers a wide range of features that help users work efficiently with text. It allows users to count words and characters accurately, remove unnecessary extra spaces, and convert text into uppercase or lowercase formats. Additionally, TextUtils calculates the estimated reading time of the entered text, making it especially useful for writers, bloggers, students, and professionals.
TextUtils has been carefully designed with usability and accessibility in mind. The clean interface ensures that users can focus entirely on their content without distractions. The application supports both light mode and dark mode, enhancing visual comfort and reducing eye strain.
One of the key advantages of TextUtils is its browser compatibility and privacy-focused approach. All text processing happens directly in the browser, ensuring that user content remains private, secure, and under complete control at all times.
Overall, TextUtils is a practical and efficient tool for anyone who works with text regularly. Its simplicity, performance, and thoughtful design make it an essential utility for everyday text analysis.


CodeHeave
Home
About Us
About TextUtils
TextUtils is a modern and user-friendly text utility web application designed to simplify everyday text processing tasks. In today's digital world, text plays a vital role in communication, documentation, and content creation. TextUtils provides a reliable platform where users can quickly analyze and transform text without the need for complex software or technical knowledge.
The application offers a wide range of features that help users work efficiently with text. It allows users to count words and characters accurately, remove unnecessary extra spaces, and convert text into uppercase or lowercase formats. Additionally, TextUtils calculates the estimated reading time of the entered text, making it especially useful for writers, bloggers, students, and professionals.
TextUtils has been carefully designed with usability and accessibility in mind. The clean interface ensures that users can focus entirely on their content without distractions. The application supports both light mode and dark mode, enhancing visual comfort and reducing eye strain.
One of the key advantages of TextUtils is its browser compatibility and privacy-focused approach. All text processing happens directly in the browser, ensuring that user content remains private, secure, and under complete control at all times.
Overall, TextUtils is a practical and efficient tool for anyone who works with text regularly. Its simplicity, performance, and thoughtful design make it an essential utility for everyday text analysis.

References:

1. React.js Official Documentation – https://react.dev
2. CodewithHarry
3. TutorialsPoint, W3Schools – React and JavaScript tutorials
4. Comparative study of Notepad, Google Docs, and MS Word

The Text Editor Website demonstrates how a modern framework like React can be used to create practical and user-friendly applications. The project balances simplicity with necessary functionality, providing essential features such as formatting tools and theme customization. It runs efficiently across devices and is scalable for future improvements. The project thus proves that web-based tools can successfully replace traditional software for basic editing tasks.

Conclusion

The Text Editor Website demonstrates how a modern framework like React can be used to create practical and user-friendly applications. The project balances simplicity with necessary functionality, providing essential features such as formatting tools and theme customization. It runs efficiently across devices and is scalable for future improvements. The project thus proves that web-based tools can successfully replace traditional software for basic editing tasks.

Acknowledgment & Thank You

We sincerely express our gratitude to **Dr. Jitendra Chaudhary**, Associate Professor, School of Computing, **Graphic Era Hill University, Bhimtal**, for his invaluable guidance and support throughout this project.

We also thank **Graphic Era Hill University** for providing us with the resources and platform to work on this project.

Lastly, we would like to extend our heartfelt appreciation to our friends, family, and peers for their constant encouragement and motivation.

Thank You!